

Data Engineering Fundamentals

Document ID	DOC003
Project	Data Engineering
Category	Data Engineering
Batch	batch_1

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Role of Data Engineering

Data engineering focuses on designing and operating systems that collect, store, transform, and deliver reliable data for analytics, reporting, machine learning, and operational use. A data engineer considers data correctness, scalability, maintainability, lineage, cost, security, and the needs of downstream consumers.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

ETL and ELT

ETL extracts data, transforms it before final loading, and then loads the prepared result into a target system. ELT extracts and loads raw or lightly processed data first, then performs transformations using the target platform's compute. Modern cloud platforms often use ELT because scalable warehouse or lakehouse compute can transform large datasets after ingestion. Neither approach is universally better; architecture depends on systems, governance, latency, and workload requirements.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Batch and Streaming

Batch processing handles bounded groups of records at intervals, such as hourly files or daily transactions. Streaming processes continuously arriving events with low latency. Batch systems are often simpler and sufficient when immediate results are unnecessary. Streaming introduces concepts such as event time, checkpoints, late data, state, and delivery semantics.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Data Lake, Warehouse and Lakehouse

A data lake stores large volumes of structured and unstructured data, often in open file formats. A data warehouse organizes curated structured data for analytics with strong SQL capabilities and governance. A lakehouse combines lake-style storage with warehouse-style management features such as transactional tables, schema controls, governance, and performant analytics.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Data Quality and Observability

Useful data-quality checks include completeness, uniqueness, validity, consistency, timeliness, and referential integrity. Pipeline monitoring should make failures and unexpected changes visible. Row counts, rejected records, processing durations, schema changes, and freshness indicators can provide practical evidence that a pipeline behaved as expected.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Idempotency and Incremental Design

An idempotent pipeline can be rerun without unintentionally duplicating results. Incremental systems avoid processing unchanged data repeatedly. Common techniques include watermarks, source modification timestamps, content hashes, processed-file registries, change data capture, and deterministic merge keys. The simplest mechanism that reliably fits the source should usually be preferred.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

End of document.